Stats 21 - HW 2

Fiona Xu

The questions have been entered into this document. You will modify the document by entering your code.

Make sure you run the cell so the requested output is visible. Download/export it as an HTML file and then print to PDF. Be sure to tag your pages on Gradescope.

Academic Integrity

The homework is practice for building your own understanding. Work through the problems, check your reasoning, and use permitted help to support your learning.

Replace `[Your Name]` to acknowledge the following:

> I, Fiona Xu, agree to follow the course academic integrity policies and the Stats 21 AI and Collaboration Policy. My submitted work reflects my own thinking and understanding, and I can explain my code and results.

Acknowledgments and AI disclosure: Give specific credit for people or sources you relied on, and disclose any AI assistance, including hints, debugging, or editing. Briefly state which problems and how each helped, or write "None".

> I used AI to debug 7.9.2, when I accidentally wrote the docstring text after my function body instead of before it. I also used it to identify the problem with my output in the last problem, when my outputs were outputting the individual words instead of the word pairs as tuples.

Reading

- *Think Python*, 3rd edition: Chapters 5 through 9, focusing on conditions, return values, iteration, strings, and lists. Recursion is optional for this homework.
- Chapter 7, Sections 7.2 and 7.7 explain reading the word file and running doctests.

Reading is important! Alternate between reading a section and working on exercises. Several exercises below are adapted from the 2nd edition; their complete prompts and links are supplied here, so you do not need a separate copy of that edition.

Additional reference:

- Python string methods

Working on this homework

Use loops or recursion where appropriate; recursion is not required. For each function, return the requested result unless the prompt explicitly asks you to print. You may assume inputs meet the stated requirements; you do not need to add input-validation code.

The supplied test calls include expected results in comments. Predict a result, run your completed function, and compare. Keep the existing tests and add your own where requested. Brief written understanding checks have a place for your response.

Textbook Chapter 5 Problems

Modified Exercise 5.14.2

https://allendowney.github.io/ThinkPython/chap05.html#exercise

In [1]:
```
import time
```

In [2]:
```
time.time()
```

Out[2]:
```
1792626064.915814
```

Write a function `now()` that reads the current time once and prints the UTC time of day in hours, minutes, and seconds, plus the number of complete days since the Unix epoch (January 1, 1970, 00:00:00 UTC). It does not need to return a value.

For example, the format should look like:

`Current time is: 15:25:47. It has been 18370 days since the epoch.`
Your numbers will reflect the time when you run the function, not the example above. Use `int()` to drop the fractional seconds, and use integer division and remainders to separate days, hours, minutes, and seconds. You do not need to determine years or months.

Build the function incrementally: first find complete days, then the time remaining within the current day. Check that hours are between 0 and 23 and minutes and seconds are between 0 and 59.

In [3]:
```
def now():
    total = int(time.time())
```

```
    days = total // 86400
    hours = (total % 86400) // 3600
    minutes = (total % 3600) // 60
    seconds = total % 60

    print(f"Current time is: {hours}:{minutes}:{seconds}. It has been {days}
```

In [4]:
```
now()
```

```
Current time is: 23:41:4. It has been 20747 days since the epoch.
```

Textbook Chapter 6 Problems

Modified Exercise 6.12.5: Greatest common divisor

https://allendowney.github.io/ThinkPython/chap06.html#id3

The greatest common divisor (GCD) of `a` and `b` is the largest positive integer that divides both with no remainder. Assume `a` and `b` are nonnegative integers and at least one is positive.

When `b > 0`, let `r` be the remainder when `a` is divided by `b`. Then `gcd(a, b) = gcd(b, r)`. When `b = 0`, the result is `a`.

Write `gcd(a, b)` to return the GCD. Use a loop or recursion; either is acceptable.

In [5]:
```
def gcd(a, b):
    if b > 0:
        r = a % b
        return gcd(b, r)
    elif b == 0:
        return a
```

In [6]:
```
gcd(21, 7)  # Expected: 7
```

Out[6]: 7

In [7]:
```
gcd(42, 28)  # Expected: 14
```

Out[7]: 14

In [8]:
```
gcd(105, 140)  # Expected: 35
```

Out[8]: 35

In [9]:
```
gcd(7, 0)  # Expected: 7
```

Out[9]: 7

In [10]:
```
gcd(0, 7)  # Expected: 7
```

Out[10]: 7

In [11]:
```
gcd(14, 25)  # Expected: 1
```

Out[11]: 1

Understanding check: In 1–3 sentences, explain why your loop or recursive calls eventually stop. What happens when the second argument is zero?

The recursive calls stop when the second argument, or the remainder after dividing the previous two arguments, is zero because it bypasses the first condition of "the second argument is greater than zero" and goes to the else condition of returning the first argument. Since the remainder is always smaller than b and never negative, the argument must eventually reach zero.

Modified Exercise 6.4 from *Think Python*, 2nd edition: Powers

Write `is_power(a, b)` to return `True` if `a = b ** k` for some nonnegative integer `k`, and `False` otherwise. Assume `a` and `b` are integers with `a >= 1` and `b >= 2`.

Exponent zero counts: `1` is a power of every allowed base because `b ** 0 = 1`. Use a loop or recursion; neither approach is required. Use integer arithmetic so that divisibility tests and intermediate values remain exact.

In [12]:
```
def is_power(a, b):
    if a == 1:
        return True
    if a % b == 0:
        return is_power(a // b, b)
    else:
        return False
```

In [13]:
```
is_power(1024, 2)  # Expected: True
```

Out[13]: True

In [14]:
```
is_power(6561, 3)  # Expected: True
```

Out[14]: True

In [15]:
```
is_power(4374, 3)  # Expected: False
```

Out[15]: False

In [16]:
```
is_power(768, 2)  # Expected: False
```

Out[16]: False

In [17]:
```
is_power(1, 2)  # Expected: True
```

Out[17]: True

In [18]:
```
is_power(2, 2)  # Expected: True
```

Out[18]: True

In [19]:
```
is_power(2, 3)  # Expected: False
```

Out[19]: False

In [20]:
```
is_power(36, 6)  # Expected: True
```

Out[20]: True

In [21]:
```
is_power(12, 6)  # Expected: False
```

Out[21]: False

Understanding check: In 1–3 sentences, explain how your function handles `a = 1` and why it eventually stops for a value that is not a power of the base.

My function explicitly handles a = 1 cases by first checking whether such a condition is true, and if it is, immediately returning "True" as per the instructions. If a is not a power of b, repeated division by b shrinks a until a % b is no longer 0, at which point the function returns "False", so the calls can't continue forever.

Textbook Chapter 7 Problems

Running the docstring examples

A **doctest** puts a function call after `>>>` inside a docstring, followed by the expected result on the next line. Calling the function normally does not run these tests.

Run the supplied helper below once. After completing each function, run its `run_doctests(...)` cell. With verbose output, each successful example is marked `ok`; otherwise you will see the expected and actual results. You do not need to rewrite the helper or understand its `globals()` and `__name__` details for this assignment.

In [22]:
```
# Supplied test helper. Run this cell once; do not rewrite it.
from doctest import run_docstring_examples


def run_doctests(function):
```

```
    """Run the examples in a function's docstring and show their results."""
    run_docstring_examples(function, globals(), name=function.__name__, verb
```

Exercise 7.9.2: Forbidden letters

https://allendowney.github.io/ThinkPython/chap07.html#exercise

Write `uses_none(word, forbidden)` to return `True` if the word contains none of the forbidden letters, and `False` otherwise. Assume both inputs contain only lowercase English letters; either input may be empty.

Complete the function so that its supplied doctests pass. Add at least one more doctest of your own and run the tests with the supplied helper.

In [23]:
```
def uses_none(word, forbidden):
    """Check whether a word avoids all forbidden letters.

    >>> uses_none('banana', 'xyz')
    True
    >>> uses_none('apple', 'efg')
    False
    >>> uses_none('excitement', 'abc')
    False
    """

    for char in word:
        if char in forbidden:
            return False
    return True
```

In [24]:
```
uses_none('banana', 'xyz')  # Expected: True
```

Out[24]:
```
True
```

In [25]:
```
uses_none('apple', 'efg')  # Expected: False
```

Out[25]:
```
False
```

In [26]:
```
uses_none('', 'abc')  # Expected: True
```

Out[26]:
```
True
```

In [27]:
```
uses_none('banana', '')  # Expected: True
```

Out[27]:
```
True
```

In [28]:
```
run_doctests(uses_none)  # Run after completing the function and adding your
```

```
Finding tests in uses_none
Trying:
    uses_none('banana', 'xyz')
Expecting:
    True
ok
Trying:
    uses_none('apple', 'efg')
Expecting:
    False
ok
Trying:
    uses_none('excitement', 'abc')
Expecting:
    False
ok
```

Exercise 7.9.3: Required letters

https://allendowney.github.io/ThinkPython/chap07.html#id1

Write `uses_all(word, required)` to return `True` if the word contains every required letter at least once, and `False` otherwise. Assume both inputs contain only lowercase English letters; either input may be empty.

Repeated letters in `required` are still presence requirements: `uses_all('ab', 'aab')` is `True`. An empty `required` string has no unmet requirements, so the result is `True`.

Complete the function so that its supplied doctests pass. Add at least one more doctest of your own and run the tests with the supplied helper.

In [29]:
```
def uses_all(word, required):
    """Checks whether a word uses all required letters.

    >>> uses_all('banana', 'ban')
    True
    >>> uses_all('apple', 'api')
    False
    >>> uses_all('sickness', 'med')
    False
    """
    for char in required:
        if char not in word:
            return False
    return True
```

In [30]:
```
uses_all('banana', 'ban')  # Expected: True
```

Out[30]:
```
True
```

In [31]:
```
uses_all('apple', 'api')  # Expected: False
```

Out[31]:
```
False
```

In [32]:
```
uses_all('', '')  # Expected: True
```

Out[32]:
```
True
```

In [33]:
```
uses_all('', 'a')  # Expected: False
```

Out[33]:
```
False
```

In [34]:
```
uses_all('ab', 'aab')  # Expected: True
```

Out[34]:
```
True
```

In [35]:
```
run_doctests(uses_all)  # Run after completing the function and adding your
```

```
Finding tests in uses_all
Trying:
    uses_all('banana', 'ban')
Expecting:
    True
ok
Trying:
    uses_all('apple', 'api')
Expecting:
    False
ok
Trying:
    uses_all('sickness', 'med')
Expecting:
    False
ok
```

Problems from Think Python 2nd Ed. Chapter 9

The local word file

`words.txt` is supplied beside this notebook in the repository's `homework/HW2` folder. No download is needed. Open and run the notebook with that folder as the working folder so that `open('words.txt')` reads the local file. If you see a `FileNotFoundError`, check the working folder rather than changing to an internet source.

Each file-loop iteration supplies one line as a string, including its trailing newline. Use `strip()` to get the word before counting characters or testing its letters. Close the file when you finish reading. The supplied example below previews one line.

In [36]:
```
# Supplied file preview; it does not solve either exercise.
fin = open("words.txt")
first_line = fin.readline()
print(repr(first_line))  # Make the newline visible.
print(first_line.strip())
fin.close()
```

```
'aa\n'
aa
```

Exercise 9.1: Long words

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 84.

Write and run a script that reads the local `words.txt` and prints only the words with **more than 20 characters**, after stripping whitespace. A word with exactly 20 characters should not be printed.

In [37]:
```
fin = open("words.txt")
for line in fin:
    word = line.strip()
    if len(word) > 20:
        print(word)
fin.close()
```

```
counterdemonstrations
hyperaggressivenesses
microminiaturizations
```

Exercise 9.2: Words without `e`

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 84.

Write `has_no_e(text)` to return `True` if the text does not contain the lowercase letter `e`, and `False` otherwise. This is a case-sensitive check, as in class. All words in the supplied file are lowercase. Any appropriate string operation is allowed.

In [38]:
```
def has_no_e(text):
    if "e" not in text:
        return True
    else:
        return False
```

In [39]:
```
has_no_e("hello")  # Expected: False
```

Out[39]: False

In [40]:
```
has_no_e("quit")  # Expected: True
```

Out[40]: True

In [41]:
```
has_no_e("")  # Expected: True
```

Out[41]: True

In [42]:
```
has_no_e("E")  # Expected: True: uppercase E is not lowercase e.
```

Out[42]: True

Use your function in a script that reads `words.txt`. Print both the number of words without `e` and their proportion among **all words in the file**. The proportion should be between 0 and 1, not a percentage. Count the denominator while reading rather than hard-coding a word count.

In [43]:
```
fin = open("words.txt")
count = 0
line_count = 0

for line in fin:
    line_count = line_count + 1
    if has_no_e(line) == True:
        count = count + 1
print (count, count / line_count)

fin.close()
```

```
37619 0.3306409084516946
```

Problems from Think Python 2nd Ed. Chapter 10

Modified Exercise 10.1: Sum possibly nested numeric data

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 100.

Write `nested_sum(t)` to return the sum of all numeric values in `t`. The outer input is a list. Its items may be integers, floats, or other lists, with lists nested to any finite depth. Flat lists, mixed numbers and lists, and empty lists are all valid inputs. Return `0` when there are no numeric values.

For example, `nested_sum([[1, 2], [3], [4, 5, 6]])` returns `21`. Do not change the original list or any of its nested lists.

A loop-based or recursive solution is acceptable. For a loop-based approach, you may use an extra list to keep track of items still to process. `isinstance(item, list)` returns `True` when an item is a list; in this problem, a non-list item is numeric. You do not need to handle strings, Booleans, or cyclic lists.

In [44]:
```
def nested_sum(t):
    total = 0
    copy = t[:]
    while copy:
        item = copy.pop()
        if isinstance(item, list):
            for inner in item:
                copy.append(inner)
        else:
            total += item
    return total
```

In [45]:
```
t = [1, 2]
nested_sum(t)  # Expected: 3
```

Out[45]: 3

In [46]:
```
t = [[1, 2], [3], [4, 5, 6]]
nested_sum(t)  # Expected: 21
```

Out[46]: 21

In [47]:
```
x = [[1, 2, [3]], 4, 5, 6, [7], 8]
nested_sum(x)  # Expected: 36
```

Out[47]: 36

In [48]:
```
t = [[[1, 2, [3]], [4, [5, 6, [7]], 8]]]
nested_sum(t)  # Expected: 36
```

Out[48]: 36

In [49]:
```
nested_sum([])  # Expected: 0
```

Out[49]: 0

In [50]:
```
nested_sum([[], [[], []]])  # Expected: 0
```

Out[50]: 0

In [51]:
```
nested_sum([[1, -2], [], 3.5, [[-0.5]]])  # Expected: 2.0
```

Out[51]: 2.0

In [52]:
```
# A copy of the outer list alone cannot detect changes to a shared inner lis
import copy
```

```
original_nested = [[1, 2], [3, [4]]]
before_nested = copy.deepcopy(original_nested)
print(nested_sum(original_nested))  # Expected: 10
print(original_nested == before_nested)  # Expected: True
```

```
10
True
```

Exercise 10.2: Cumulative sums

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 100.

Write `cumsum(t)` to accept **any finite iterable of numbers** and return a new list of cumulative sums. The item at index `i` of the result is the sum of the first `i + 1` input values, in their iteration order. For example, `cumsum([1, 2, 3])` returns `[1, 3, 6]`. Empty input returns `[]`.

Lists, ranges, and iterators are valid inputs; all supplied values are numeric and there are no nested lists. Do not assume the input supports indexing or `len()`. Do not modify a list passed as input. Consuming a one-pass iterator while reading its values is expected.

In [54]:
```
def cumsum(t):
    result = []
    total = 0
    for num in t:
        total += num
        result.append(total)
    return result
```

In [55]:
```
cumsum([1, 2, 3, 4])  # Expected: [1, 3, 6, 10]
```

Out[55]:
```
[1, 3, 6, 10]
```

In [56]:
```
cumsum(range(12))  # Expected: [0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66]
```

Out[56]:
```
[0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55, 66]
```

In [57]:
```
cumsum([])  # Expected: []
```

Out[57]:
```
[]
```

In [58]:
```
cumsum([7])  # Expected: [7]
```

Out[58]:
```
[7]
```

In [59]:
```
cumsum([3, -5, 2])  # Expected: [3, -2, 0]
```

Out[59]:
```
[3, -2, 0]
```

`iter([1, 2, 3])` supplies those values once, in order. This test checks that the function can loop over an input without indexing it.

In [60]:
```
cumsum(iter([1, 2, 3]))  # Expected: [1, 3, 6]
```

Out[60]:
```
[1, 3, 6]
```

In [61]:
```
values = [3, -5, 2]
before = values.copy()
print(cumsum(values))  # Expected: [3, -2, 0]
print(values == before)  # Expected: True
print(values)  # Expected: [3, -5, 2]
```

```
[3, -2, 0]
True
[3, -5, 2]
```

Exercise 10.6: Anagrams

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101.

Two strings are anagrams if their letters can be rearranged to match exactly. Write `is_anagram(word1, word2)` to return `True` for anagrams and `False` otherwise. Ignore letter case and ordinary spaces using `text.replace(" ", "").lower()`. Assume inputs contain only English letters and ordinary spaces.

Letter **counts** matter: matching lengths and the same distinct letters are not enough. Two empty strings count as anagrams after normalization.

In [62]:
```
def is_anagram(word1, word2):
    word1 = word1.lower().replace(" " , "")
    word2 = word2.lower().replace(" " , "")

    if sorted(word1) == sorted(word2):
        return True
    else:
        return False
```

In [63]:
```
is_anagram("hello", "o hell")  # Expected: True
```

Out[63]:
```
True
```

In [64]:
```
is_anagram("dormitory", "dirty room")  # Expected: True
```

Out[64]:
```
True
```

In [65]:
```
is_anagram("dormitory", "dirty rooms")  # Expected: False
```

Out[65]: False

In [66]:
```
is_anagram("astronomers", "moon starers")  # Expected: True
```

Out[66]: True

In [67]:
```
is_anagram("aab", "abb")  # Expected: False
```

Out[67]: False

In [68]:
```
is_anagram("aab", "aba")  # Expected: True
```

Out[68]: True

In [69]:
```
is_anagram("Dormitory", "dirty room")  # Expected: True
```

Out[69]: True

In [70]:
```
is_anagram("", "")  # Expected: True
```

Out[70]: True

Understanding check: In 1–3 sentences, explain why `"aab"` and `"abb"` are not anagrams even though they have the same length and the same distinct letters.

An anagram requires each letter in the second word to appear exactly the number of times they appear in the first word. In this case, there are two "a"s in the first string and only one "a" in the second, so they are not anagrams.

Exercise 10.7: Duplicate values

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101.

Write `has_duplicates(t)` to return `True` if any value appears more than once, and `False` otherwise. It must not modify the input list. An empty list has no duplicates.

The list may mix integers and strings but will not contain nested lists. Compare values using equality. Remember that sorting a mixed list of numbers and strings can raise an error, as shown in class.

In [71]:
```
def has_duplicates(t):
    count = []
    unique = list(set(t.copy()))
    for x in unique:
        if t.count(x) > 1:
            return True
    return False
```

In [72]:
```
has_duplicates(['a', 'b', 'c'])  # Expected: False
```

Out[72]: False

In [73]:
```
has_duplicates(['a', 'b', 'b', 'c'])  # Expected: True
```

Out[73]: True

In [74]:
```
has_duplicates(['a', 'b', 'c', 'a'])  # Expected: True
```

Out[74]: True

In [75]:
```
has_duplicates([])  # Expected: False
```

Out[75]: False

In [76]:
```
has_duplicates([1, '1', 2])  # Expected: False: the number 1 differs from th
```

Out[76]: False

In [77]:
```
# Deliberately unsorted: this check detects accidental in-place sorting.
values = [3, 1, 3]
before = values.copy()
print(has_duplicates(values))  # Expected: True
print(values == before)  # Expected: True
print(values)  # Expected: [3, 1, 3]
```

```
True
True
[3, 1, 3]
```

In [78]:
```
values = [3, 1, 2]
before = values.copy()
print(has_duplicates(values))  # Expected: False
print(values == before)  # Expected: True
```

```
False
True
```

Exercise 10.10: Binary search

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101.

Write `in_bisect(word_list, word)` to return `True` if a target word is present in a sorted list, and `False` otherwise. Use binary search, implemented with iteration or recursion. Do not replace the search with `word in word_list`, `.index()`, or a library search function.

Compare the target with the middle word. If they are equal, you have found it. Otherwise search only the half where the target could still occur, excluding the middle word that

you already checked. If no candidates remain, return `False`.

Assume `word_list` is already sorted in Python's default string order and the target and list use matching lowercase spelling. Do not change the input list. Empty lists and one-item lists must work. The supplied file has about 114,000 words, so binary search needs only about 17 middle-word comparisons in the worst case.

The supplied helper loads the local word list. You do not need to rewrite it.

In [79]:
```
# Supplied helper: use this function without rewriting it.
def make_word_list():
    """Read the local words.txt and return its words in their existing sorte
    words = []
    fin = open('words.txt')
    for line in fin:
        words.append(line.strip())
    fin.close()
    return words


t = make_word_list()
```

In [80]:
```
# define this function
def in_bisect(word_list, word):
    if len(word_list) == 0:
        return False

    i = len(word_list) // 2
    middle = word_list[i]

    if middle == word:
        return True
    elif middle > word:
        word_list = word_list[ : i]
        return in_bisect(word_list, word)
    else:
        word_list = word_list[i + 1 : ]
        return in_bisect(word_list, word)
```

In [81]:
```
in_bisect(t, "hello")  # Expected: True
```

Out[81]: True

In [82]:
```
in_bisect(t, "xyz")  # Expected: False
```

Out[82]: False

In [83]:
```
in_bisect([], "cat")  # Expected: False
```

Out[83]: False

In [84]:
```
in_bisect(["cat"], "cat")  # Expected: True
```

Out[84]: True

In [85]:
```
in_bisect(["cat"], "ant")  # Expected: False
```

Out[85]: False

In [86]:
```
in_bisect(["cat"], "dog")  # Expected: False
```

Out[86]: False

In [87]:
```
small_words = ["ant", "bee", "cat", "dog", "eel"]
in_bisect(small_words, "ant")  # Expected: True: first item.
```

Out[87]: True

In [88]:
```
in_bisect(small_words, "cat")  # Expected: True: middle item.
```

Out[88]: True

In [89]:
```
in_bisect(small_words, "eel")  # Expected: True: last item.
```

Out[89]: True

In [90]:
```
in_bisect(small_words, "bat")  # Expected: False: would fit between existing
```

Out[90]: False

In [91]:
```
in_bisect(small_words, "zzz")  # Expected: False: comes after all items.
```

Out[91]: False

Understanding check: In 1–3 sentences, explain why each unsuccessful middle-word comparison reduces the remaining candidates and how your function knows when the word is absent.

When the code fails to match the middle-word with the input word, it runs the function again, either with the lower or upper half of the remaining candidates, depending on how the value of the original middle-word compared with the input word. Thus, the function will keep splitting the remaining word list in half and evaluating which half to use based on the value comparisons of the middle and input words. Eventually, when splitting the word list in half while excluding the middle word results in a list with zero words, the function returns "False", showing that the word is absent.

Exercise 10.11: Reverse pairs, including palindromes

https://greenteapress.com/thinkpython2/thinkpython2.pdf

Page 101.

Two words form a reverse pair when one is the reverse spelling of the other. Write `reverse_pairs(word_list)` to use your `in_bisect` function and print all reverse pairs in the supplied list with words of **at least 6 letters**. Assume `word_list` is already sorted and has no duplicates. The function prints its results; it does not need to return a value.

Print each result as `(word, reversed_word)`. For two different words, include both orientations, as in the original examples: `('animal', 'lamina')` and `('lamina', 'animal')`. A palindrome is a valid self-pair and appears **once**, for example `('deified', 'deified')`. You do not need to print that self-pair a second time.

To reverse a string, `word[::-1]` reads all characters with a step of `-1`. For example, `'desserts'[::-1]` is `'stressed'`. Start by testing your function with a small sorted trial list, then run it with the full word list `t`.

In [92]:
```
# Example output includes both orientations of distinct reverse pairs:
# ('agenes', 'senega')
# ('animal', 'lamina')
# ...
# ('lamina', 'animal')
# ...
# ('senega', 'agenes')
# Palindromes appear once each, for example:
# ('deified', 'deified')
# ('redder', 'redder')
```

For a small trial list `['animal', 'deified', 'lamina', 'redder']`, the expected pairs are the four below, in any order. You can temporarily use this list while developing your script, but show the full-file results in your final submission.

```
('animal', 'lamina')
('deified', 'deified')
('lamina', 'animal')
('redder', 'redder')
```

In [93]:
```
def reverse_pairs(word_list):
    for word in word_list:
        if len(word) >= 6:
            reversed_word = word[::-1]
            if in_bisect(word_list, reversed_word):
                print((word, reversed_word))
```

In [94]:
```
# Test on a small sorted list.
small_list = ["animal", "deified", "lamina", "redder"]
reverse_pairs(small_list)
```

```
('animal', 'lamina')
('deified', 'deified')
('lamina', 'animal')
('redder', 'redder')
```

In [95]:
```
# Test on the full word list loaded above.
reverse_pairs(t)
```

```
('agenes', 'senega')
('animal', 'lamina')
('animes', 'semina')
('degami', 'imaged')
('deified', 'deified')
('deifier', 'reified')
('deliver', 'reviled')
('denier', 'reined')
('denies', 'seined')
('denned', 'denned')
('depots', 'stoped')
('derats', 'stared')
('dessert', 'tressed')
('desserts', 'stressed')
('dewans', 'snawed')
('dialer', 'relaid')
('diaper', 'repaid')
('dormin', 'nimrod')
('drawer', 'reward')
('elides', 'sedile')
('eviler', 'relive')
('gelder', 'redleg')
('halalah', 'halalah')
('hallah', 'hallah')
('imaged', 'degami')
('lamina', 'animal')
('levins', 'snivel')
('looter', 'retool')
('marram', 'marram')
('nimrod', 'dormin')
('pupils', 'slipup')
('recaps', 'spacer')
('redder', 'redder')
('redips', 'spider')
('redleg', 'gelder')
('redraw', 'warder')
('redrawer', 'rewarder')
('reflet', 'telfer')
('reflow', 'wolfer')
('reified', 'deifier')
('reifier', 'reifier')
('reined', 'denier')
('reknit', 'tinker')
('reknits', 'stinker')
('relaid', 'dialer')
('relive', 'eviler')
('remeet', 'teemer')
('rennet', 'tenner')
('repaid', 'diaper')
('repaper', 'repaper')
('repins', 'sniper')
('retool', 'looter')
('reviled', 'deliver')
('reviver', 'reviver')
('reward', 'drawer')
('rewarder', 'redrawer')
```

```
('rotator', 'rotator')
('sallets', 'stellas')
('scares', 'seracs')
('secret', 'terces')
('sedile', 'elides')
('seined', 'denies')
('selahs', 'shales')
('selles', 'selles')
('sememes', 'sememes')
('semina', 'animes')
('senega', 'agenes')
('seracs', 'scares')
('shales', 'selahs')
('skeets', 'steeks')
('sleeps', 'speels')
('sleets', 'steels')
('slipup', 'pupils')
('sloops', 'spools')
('snawed', 'dewans')
('sniper', 'repins')
('snivel', 'levins')
('snoops', 'spoons')
('spacer', 'recaps')
('speels', 'sleeps')
('spider', 'redips')
('spirts', 'strips')
('spools', 'sloops')
('spoons', 'snoops')
('sports', 'strops')
('sprits', 'stirps')
('stared', 'derats')
('steeks', 'skeets')
('steels', 'sleets')
('stellas', 'sallets')
('stinker', 'reknits')
('stirps', 'sprits')
('stoped', 'depots')
('stressed', 'desserts')
('strips', 'spirts')
('strops', 'sports')
('struts', 'sturts')
('sturts', 'struts')
('teemer', 'remeet')
('telfer', 'reflet')
('tenner', 'rennet')
('terces', 'secret')
('terret', 'terret')
('tinker', 'reknit')
('tressed', 'dessert')
('warder', 'redraw')
('wolfer', 'reflow')
```